

Python

1. Safe vs Unsafe SQL Statements

Safe (Parameterized Queries)

Look for:

```
cursor.execute("SELECT * FROM table WHERE column = ?", [value])
cursor.execute("INSERT INTO table (col1, col2) VALUES (?, ?)", [val1, val2])
```

- Uses `?` or `%s` placeholders (depending on DB driver).
- Data values are passed as a **separate argument (list or tuple)**.
- **Prevents SQL Injection.**

Unsafe (String Interpolation or f-strings)

Look for:

```
cursor.execute(f"SELECT * FROM table WHERE col = '{user_input}'")
cursor.execute("DELETE FROM table WHERE col = '%s'" % user_input)
```

- Directly embeds user input in query string.
- Vulnerable to inputs like `'; DROP TABLE users; --`.
- **SQL Injection risk!**

2. Does the query run once or in a loop?

Check if SQL queries are run:

- **In a `for` loop** → Each execution runs separately (can affect performance or commit behavior).
- **With `autocommit=True`** → Every update is permanent instantly.

If a crash happens during a loop:

- With transactions (`autocommit=False`) → Partial work can be rolled back.
- With `autocommit=True` → Partial updates will **remain**.

3. Idempotency: What happens if the function is run twice?

Look for:

- Does the code **check before INSERT/UPDATE**?
- Will it **append** something to a field like a name (`product_name || '_USA'`)?

Examples:

```
SET product_name = product_name || '_USA'
```

- This will keep appending if run multiple times.
-  Check logic to avoid duplication if that's undesired.

4. SQL Injection Attack Simulation

Attack pattern to try:

```
"something'; DROP TABLE tablename; --"
```

If the query looks like this:

```
f"SELECT * FROM table WHERE name = '{user_input}'"
```

You are in danger.

But if it looks like:

```
"SELECT * FROM table WHERE name = ?", [user_input]
```

You're safe.

5. Connection & Cursor Safety

Always check:

- Is the **connection closed** in a `finally` block?
- Are **cursors** used with `with` blocks or closed manually?

```
# Good
with conn.cursor() as cur:
    ...

# Good
try:
    ...
finally:
    if conn is not None:
        conn.close()
```

Quick Red Flags Checklist:

Red Flag	Explanation
<code>f"SELECT ... {user_input}"</code>	 SQL Injection possible
<code>"... WHERE col = '%s'" % var</code>	 String formatting in SQL is unsafe
Looping over many updates with <code>autocommit=True</code>	 Risk of partial updates
Appending to fields (<code>`name</code>	
No check before insert	 Might insert duplicates
No <code>finally</code> with <code>conn.close()</code>	 Connection may stay open on crash